

Intelligent Git Conflict Resolver & Impact Simulator

Tech Stack: Python (FastAPI, Tree-sitter, OpenAI/Langchain SDK), React (TypeScript, Tailwind, React Flow).

System Architecture

1. **User Input:** User pastes a file containing Git conflict markers into the React UI.
2. **API Ingestion:** FastAPI receives the text.
3. **Regex Extraction:** Python parses the raw text into `Current_Change` , `Incoming_Change` , and `Clean_Context` .
4. **AST Traversal:** Python uses `tree-sitter` to parse the `Clean_Context` and identify what variables or functions are affected by the conflict zone (The "Blast Radius").
5. **AI Resolution:** The conflict block and the AST dependencies are sent to an LLM via a strict prompt.
6. **Data Formatting:** The backend returns a structured JSON graph.
7. **UI Rendering:** React Flow renders the file structure, the conflict, and the blast radius as an interactive graph.

Step 1: Environment & Tooling Setup

Start by setting up your backend and frontend repositories.

Backend (Python/FastAPI)

```
mkdir conflict-resolver-backend && cd conflict-resolver-backend
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
pip install fastapi uvicorn openai tree-sitter tree-sitter-javascript pydantic
```

Frontend (React/Vite)

```
npm create vite@latest conflict-resolver-frontend -- --template react-ts
cd conflict-resolver-frontend
npm install reactflow axios tailwindcss
```

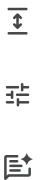
Step 2: The Parsing Engine (Python)

Before we do AI, we need to extract the conflict. Create a file named `parser.py` . You need a function that looks for Git's standard markers:

- `<<<<<<< HEAD` (Start of your current code)
- `=====` (Divider)
- `>>>>>>> incoming-branch` (End of incoming code)

Goal for this step: Write a Python function that takes a raw string and returns a Python dictionary:

```
{
  "pre_conflict_code": "...",
  "current_change": "...",
  "incoming_change": "...",
  "post_conflict_code": "..."
}
```



Step 3: The Impact Simulator (Tree-sitter)

This is the hardest but most impressive part of the project. We use `tree-sitter` to read the JavaScript/TypeScript code just like a compiler would.

Create `ast_analyzer.py`.

1. Initialize the Tree-sitter JavaScript parser.
2. Feed it the `pre_conflict_code` and `post_conflict_code`.
3. Write a Tree-sitter "Query" (which looks a bit like Lisp) to find all function declarations and variable usages.

Why this matters for your portfolio: If the incoming Git change renames a function from `fetchData` to `getUserData`, your AST analyzer will scan the rest of the file and say, *"Wait, fetchData is still being called on line 45! If you accept this merge, the file will break."* Your Python script should output a list of **Dependent Nodes** (e.g., "Function X depends on Variable Y located inside the conflict zone").

Step 4: The AI Resolution Engine

Now that you have the conflict AND the dependencies, you use AI to solve it. In `main.py` (your FastAPI entry point), construct the prompt using the OpenAI SDK.

The Golden Prompt Structure:

"You are an expert software engineer resolving a Git merge conflict.

Current Code: {current_change} **Incoming Code:** {incoming_change}

Context: The surrounding file relies on these specific variables and functions:
{ast_dependencies}

Merge these changes intelligently. Ensure that the intent of BOTH developers is preserved if possible. If the incoming change renames a variable, ensure you provide the code to update the dependent functions. Return the output as valid JSON containing `resolved_code` and `explanation`."

Step 5: The FastAPI Endpoint

Tie it all together in `main.py`. Create a `POST /api/resolve` endpoint.

1. It receives the raw text from the frontend.
2. Calls the Parser.
3. Calls the AST Analyzer.
4. Calls the AI.
5. Returns a unified JSON response containing the nodes and edges required by React Flow, plus the AI's resolved code.

Step 6: The Frontend Graph Visualizer (React Flow)

In your React app, build a dashboard.

1. **The Input:** A large text area where the user pastes the conflicted code.
2. **The Graph:** Use `reactflow` to render the backend's response.
 - Create a central, red node representing the **Merge Conflict**.
 - Create connecting nodes representing the **Functions/Variables** that will be impacted by the merge (The Blast Radius).
3. **The Resolution:** A panel on the side that displays the AI's explanation and the newly generated, safely merged code block.

Portfolio Pro-Tips

- **Mock the AI while developing:** AI API calls take 3-5 seconds. While building the React frontend, hardcode a dummy JSON response in FastAPI so you can build the UI instantly without waiting or spending API credits.
- **Focus on one language first:** Tree-sitter requires different grammars for different languages. Hardcode your app to *only* support analyzing JavaScript or TypeScript files for the MVP. You can expand later.
- **Edge Cases:** What if the conflict is just a simple formatting difference (tabs vs spaces)? Write logic in Python to auto-resolve identical logic before even asking the AI, showing you care about performance and cost optimization.